Check for updates

Original research article

International Journal of HIGH PERFORMANCE COMPUTING APPLICATIONS

The International Journal of High Performance Computing Applications
2026, Vol. 0(0) 1–17
© The Author(s) 2026
Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/10943420261446003
journals.sagepub.com/home/hpc
Sage

Enabling kilometer-scale E3SM land model simulation over North America: A new integrated framework solution

Dali Wang[1], Chen Wang[2], Qinglei Cao[3], Jayesh Krishna[4], Weijian Zheng[4], Peter Schwartz[1], Fengming Yuan[1], Daniel Ricciuto[1], Peter Thornton[1] and Shih-Chieh Kao[1]

Abstract

This study introduces a novel framework designed to enhance the performance, scalability, and portability of the kilometer-scale E3SM Land Model (km-ELM) within the E3SM modeling infrastructure. By seamlessly integrating cutting-edge data tools, we address existing challenges such as slow performance, limited scalability, and difficulties in software integration in current data-driven ELM simulation over large geographic areas. Our innovative approach leverages the KiloCraft data toolkit to generate unified inputs for simulations ranging from a single-cite case, to a 72,083-cell regional case to a continental configuration encompassing 21.6 million land grid cells at a 1 km × 1 km resolution. We conduct extensive strong- and weak-scaling experiments on three state-of-the-art supercomputers, utilizing up to 100,800 CPU cores across 2400 compute nodes to evaluate end-to-end metrics including wall-clock time, simulation-years-per-day (SYPD), initialization costs, and I/O throughput. Our results reveal the land (LND) component's efficient scaling, demonstrating near-ideal weak scaling and strong-scaling parallel efficiencies reaching up to 87% at 50,400 cores. We confirm portability and reproducibility through bitwise-equivalent outputs across different machines using identical inputs over supported machines. Notably, at extreme scales, we identify I/O as a critical bottleneck and that leads to effective solution with the SCORPIO/ADIOS stack. Collectively, these findings validate the deployment of km-ELM at a continental scale with high parallel efficiency and provide essential guidance on configuration, decomposition, and I/O settings for optimized kilometer-scale land simulations in E3SM. This work emphasizes the innovative design and practical solutions that enhance the operational capabilities of km-ELM, focusing on software performance and scalability while leaving detailed scientific evaluations of simulated land processes for future investigations.

Keywords

E3SM land model (ELM), km-ELM, scalability, performance evaluation, software design

1. Introduction

Earth System Models (ESMs) are essential tools for advancing our understanding of the Earth system and its response to both natural variability and anthropogenic influences. These models integrate components such as the atmosphere, oceans, and land surface, along with their dynamic interactions, to provide valuable insights into past, present, and future Earth system conditions on infrastructure and energy security. The development and application of ESMs demand substantial computational resources and specialized expertise.

The Energy Exascale Earth System Model (E3SM), funded by the US Department of Energy (DOE), was developed to align with the nation's scientific priorities and the DOE's mission objectives. E3SM distinguishes itself from many other ESMs by emphasizing energy-related research, including investigations into the impacts of Earth system change on energy systems such as water availability, severe weather events, and sea level rise (Golaz et al., 2019). A key

[1]Environmental Sciences Division, Oak Ridge National Laboratory, Oak Ridge, TN, USA
[2]College of Computing and Data Science, Nanyang Technological University, Singapore, Singapore
[3]Department of Computer Science, Saint Louis University, MO, USA
[4]Mathematics and Computer Science Division, Argonne National Laboratory, Lemont, IL, USA

Corresponding author:
Dali Wang, Environmental Sciences Division, Oak Ridge National Laboratory, MS 6301, PO Box 2008, Oak Ridge, TN 37831, USA.
Email: wangd@ornl.gov

advantage of E3SM is its high-resolution capability, which allows for improved representation of smaller-scale processes like regional weather patterns and ocean currents. The model is optimized for exascale supercomputers, enabling complex simulations that would be computationally prohibitive on earlier systems, and thereby supporting more detailed and accurate Earth system predictions (Caldwell et al., 2019).

The E3SM Land Model (ELM) (Burrows et al., 2020) provides detailed representations of vegetation, soil moisture, surface temperature, and land-use changes. ELM interacts with other E3SM components, such as the atmosphere and oceans, to gain insights into how these elements influence each other and the overall Earth system. Key processes modeled by ELM include energy and water exchange, nutrient cycling, vegetation dynamics, and human activities such as agriculture and deforestation. ELM is essential for understanding surface conditions that influence atmospheric circulation and long-term patterns, making it a valuable tool to improve infrastructure resilience and ensure energy security.

Historically, the majority of ESM simulations employ grids larger than 0.5 × 0.5 degrees (Soares et al., 2024). The Energy Exascale Earth System Model version 1 (E3SMv1) operates at two distinct horizontal resolutions: a standard resolution of approximately 100 km (roughly 1 degree) grid spacing for both land and atmosphere, and a high-resolution version with 25 km (0.25 degrees) grid spacing, enabling more detailed representations of ocean eddies and atmospheric storms to improve understanding of planetary circulation and its implications for human activities (Caldwell et al., 2019). E3SM is advancing kilometer-scale models to fully exploit exascale supercomputers for high-resolution environmental simulations. This effort includes the development of the km-ELM, along with other E3SM components such as the Atmosphere and Ocean models, all designed for exascale performance (Donahue et al., 2024; Petersen et al., 2025).

The availability of high-fidelity data products, such as meteorological forcing and detailed soil property datasets (Thornton et al., 2021; Han et al., 2023; Commission for Environmental Cooperation (CEC), 2024), further enhances the capability of kilometer-scale ELM (km-ELM) to capture fine-scale geographic heterogeneity, land–atmosphere interactions, and extreme weather events with improved realism. These detailed inputs not only provide stronger physical constraints but also enable a more accurate representation of subgrid processes critical to land surface modeling. However, the substantial increase in spatial resolution leads to exponential growth in computational load, memory footprint, and I/O volume, thereby imposing significant challenges on scalability, data throughput, and long-term storage. Addressing these demands requires advanced HPC resources and efficient data management strategies.

To meet these demands, we need advanced high-performance computing (HPC) resources and effective data management. This paper focuses solely on the computational performance of the kilometer-scale E3SM Land Model (km-ELM), not on the scientific accuracy of its outputs. We evaluate km-ELM in offline mode, emphasizing software performance and scalability across modern HPC architectures. Our analysis highlights key metrics such as parallel efficiency, throughput (simulation-years-per-day), initialization costs, and input/output (I/O) behavior, using standardized configurations and datasets. We do not assess the scientific validity of simulated fields over North America; our main goal is to enhance computational efficiency. To tackle these challenges, we have developed innovative techniques to optimize model configuration and improve data handling. Our contributions significantly enhance the capabilities for extreme-scale km-ELM simulations, summarized as follows:

- We configure km-ELM with CIME and the KiloCraft toolkit to enable standardized kilometer-scale runs from a single grid cell to a 21.6-million–cell North America domain.
- We quantify strong and weak scaling on three HPC systems using up to 100,800 CPU cores (2400 nodes), reporting wall-clock time, simulation-years-per-day (SYPD), initialization cost, and I/O throughput, and we verify portability via bitwise-equivalent outputs across machines.
- We identify bottlenecks at extreme scale, particularly during initialization and write I/O via SCORPIO/PnetCDF. We have achieved exceptional I/O performance by utilizing an alternative backend (Wang et al., 2025) on Frontier, which complements the end-to-end flow of the kmELM framework.

To the best of our knowledge, this is the first work to perform full-capability km-ELM simulations with dynamic biogeochemical functionality across diverse computational domains, notably encompassing 21.6 million land grid cells representing approximately 21.5 million square kilometers of North America at a 1 km × 1 km resolution.

The structure of this paper is organized as follows: the Related Work section reviews prior research pertinent to our study, while the Framework section delineates the methodological approach. Subsequently, the Results and Discussion are presented, followed by conclusions and suggestions for future research.

2. Related work

ESMs have evolved significantly, integrating components such as the atmosphere, oceans, land surface, cryosphere, and biosphere to simulate the Earth system. Notable examples include E3SM (Golaz et al., 2019), developed by the

U.S. DOE to investigate energy-related environmental impacts, and the Community Earth System Model (CESM) (Danabasoglu et al., 2020), developed by the National Center for Atmospheric Research (NCAR), which facilitates comprehensive Earth system simulations. The Geophysical Fluid Dynamics Laboratory (GFDL) ESMs (Dunne et al., 2020) focus on ecological variability and societal impacts, while the UK Earth System Model (UKESM) (Sellar et al., 2019) emphasizes atmospheric chemistry and land-use dynamics. The Australian Community Earth System Simulator (ACCESS) (Puri, 2012) is developed to understand unique aspects of Australia's past and future ecosystem.

High-resolution simulations powered by advanced computing are fundamental to state-of-the-art ESMs. Currently, E3SM is the only model optimized for exascale supercomputers, enabling kilometer-scale simulations that capture critical processes such as mesoscale weather patterns and localized hydrological cycles. This high spatial and temporal resolution leads to more accurate forecasts and enhances our understanding of regional environmental variability and change, which is essential for effective policy-making and resource management (Donahue et al., 2024). The E3SM atmosphere model, particularly its SCREAM (Simple Cloud-Resolving E3SM Atmosphere Model) version, enables convection-permitting simulations at resolutions down to 3.25 km, improving the representation of phenomena such as precipitation and cloud formation. The ocean model, along with the sea ice component, also benefits from higher resolution, resulting in more realistic simulations of oceanic circulation and sea ice dynamics (Petersen et al., 2025) at a similar resolution.

The ELM software comprises approximately half a million lines of code, featuring highly specialized data types, around 2000 globally accessible multidimensional arrays, and over 1000 subroutines (Xu et al., 2017; Zheng et al., 2019). Recent advances in km-ELM include the development of the ultrahigh-resolution ELM (uELM) framework, which enables kilometer-scale land surface modeling (Wang et al., 2022), as well as its application to localized regions (Gu et al., 2025; Hao et al., 2022; Yuan et al., 2023). Efforts have also been made to port uELM and establish a functional unit testing (FUT) framework to support development and verification (Schwartz et al., 2022a, 2022b). However, earlier km-ELM simulations were constrained to small domains (typically fewer than 10,000 grid cells) and ran on small HPC systems with no more than 10 compute nodes. In contrast, this study presents the first full-capability km-ELM simulations conducted across diverse computational domains at an ultrahigh resolution of 1 km × 1 km over North America, encompassing more than 21.5 million land grid cells.

3. Methodological framework

The following subsections outline our methodological framework for km-ELM: model configuration, input data generation, numerical experiments to assess ELM simulation readiness, and results validation and performance profiling.

3.1. Km-ELM configuration and simulation

The Common Infrastructure for Modeling the Earth (CIME)[1] is a crucial framework used by the E3SM project. CIME provides a standardized toolkit, procedures, and workflows that streamline model development across various components. In our work, we utilize CIME scripts to configure, build, and submit batch jobs for the km-ELM simulation. Currently, the km-ELM consists of an active land model, a data atmosphere model, and a coupler component, integrated through CIME.

3.1.1. Component configuration and dataflow. The experiment employs a designated component setting (`I1850uELMCNPRDCTCBC`). Within this configuration, the active E3SM land simulation is driven by predefined atmospheric forcing to achieve steady states for transient km-ELM simulations starting from the Industrial Revolution (1850 AD). User-defined forcing data are retrieved from the parallel file system via a data atmosphere model,[2] and data exchange is enabled through a coupler, as illustrated in Figure 1. The land model is configured to simulate

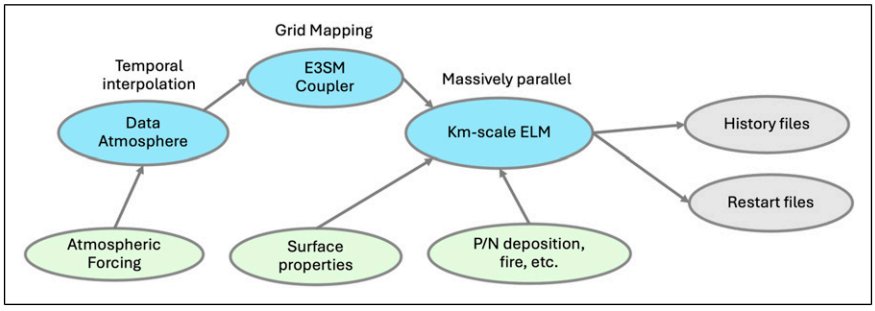


Figure 1. Km-ELM data flow and simulation configuration, which requires three groups of datastreams: atmospheric forcing, land surface properties, and others (e.g., nitrogen and phosphorus (N/P) deposition).

biophysical and biochemical processes encompassing energy, water, and nutrient dynamics under resource demand (RD), and includes a soil organic matter decomposition model (converging trophic cascade (CTC)).

The experiments utilize custom grids specified in the input domain files, including a 1 km × 1 km resolution grid covering North America. A new setup was developed for the data atmosphere model to retrieve user-defined atmospheric forcing data from disk. Within the km-ELM framework, both the atmospheric and land models utilize the same domain file, thereby enhancing simulation efficiency by eliminating the need for spatial grid conversion and interpolation. In instances where atmospheric forcing data are unavailable at the necessary time intervals for the land model, the atmospheric model conducts temporal interpolation to ensure synchronization during the simulation.

The new km-ELM setup has been successfully merged into the main E3SM repository,[3] is actively maintained by the E3SM software infrastructure team, ensuring that the community can readily access and utilize this advanced capability, empowering researchers and practitioners to leverage km-ELM in their studies and applications without the need for extensive setup or configuration.

3.1.2. Computational domain, domain decomposition, and parallel execution. The ELM computational domain employs a Lambert Conformal Conic (LCC) projection derived from the Daymet data product (Thornton et al., 2021). This conic map projection, centered at a latitude of 42.5, preserves shape accuracy and provides an effective representation of North America, supporting precise mapping in meters. The ELM domain also adopts the WGS84 global geodetic reference system to facilitate data exchange with other Earth system components.

The km-ELM uses MPI (Snir, 1998) for parallel processing, distributing a fixed number of land grid cells among MPI processes using static domain decomposition. Three static partitioning schemes (i.e., round-robin, block round-robin, and block partition), along with flexible user-defined decomposition strategies, are implemented (Gu et al., 2025). Each MPI process retains a complete copy of the ELM domain information, including all subdomain dimensions and sizes; however, memory is allocated only for the subdomain assigned to that process. Within each MPI process, the ELM model acquires atmospheric forcing data from the atmospheric model, utilizes surface characteristics and land-use datasets to initialize land grid cells within its designated subdomain, and then conducts parallel simulations across those grid cells independently. Depending on the simulation domain size and hardware configuration, km-ELM can run on CPUs or GPUs using OpenACC (Schwartz et al., 2022b). This study exclusively utilizes CPU resources, with GPU deployment planned for future work.

3.1.3. I/O and data management. ELM provides the capability to generate time-averaged history outputs and restart files at user-defined intervals, which may be timestep-based, hourly, daily, monthly, annually, or disabled entirely. It also allows users to customize output variables beyond the default set.

In this study, simulation outputs are saved as history files (Figure 1), which can later be processed and analyzed using the km-ELM data management and analysis component, developed based on VISIT (Ayachit, 2015) and QGIS (QGIS Development Team, 2009). Additionally, a restart function is enabled to periodically save files for all active Earth system components, including the data atmosphere model, the coupler, and the land model. These restart files enable simulations to resume from specific time points, which is particularly useful for long-term simulations or recovery from system failures.

The size of history files depends on several factors, including output frequency, number of grid cells, the number of variables per grid cell level, and their data types. In contrast, the size of restart files is directly proportional to the number of grid cells and their subgrid elements, as well as the number and type of stored variables across all levels.

The data produced by ELM is not decomposed in memory across compute processes in an "I/O-friendly" manner. To improve I/O performance, km-ELM employs a specialized parallel I/O library, SCORPIO (Hartnett and Edwards, 2021; Krishna et al., 2025), to manage data access. SCORPIO rearranges the data distributed across multiple processes before passing it to backend I/O libraries such as PnetCDF (Li et al., 2003) and ADIOS (Gu et al., 2018), which handle writing the data to the file system. These I/O libraries may further enhance performance by aggregating operations from multiple MPI processes into smaller groups, known as aggregators (Corbett et al., 1996), which interact directly with the underlying parallel file systems, such as Lustre (SUN, 2007) and GPFS (Schmuck and Haskin, 2002).

3.2. Km-ELM input data generation

Drawing from prior research in large-scale data processing (Wang et al., 2023), we have developed a comprehensive data toolkit named KiloCraft. This toolkit is designed to facilitate the generation of appropriate input datasets (such as forcing, surface properties, and land use) for km-ELM simulations.

KiloCraft provides essential functionalities for generating domain, forcing, and surface data at user-specified resolutions within a defined computational domain, such as North America in this study. It also includes features for data subsetting in regions of interest, duplicating input datasets, performing sanity checks, validating across different data products, and visualizing the data.

3.2.1. Domain over North America. We extract domain information from the Daymet product, including grid dimensions, variable values, and coordinate data. Grid cell IDs are computed, land grid cell masks are created, and the area and land fraction for each grid cell are determined. Coordinate conversions are performed between the LCC projection system and the WGS84 geodetic reference system, and 1D and 2D ELM domains are generated over North America. These steps are critical for defining the computational domain and preparing meteorological data for subsequent simulations.

The resulting km-ELM domain provides a distinct 1 km × 1 km resolution across North America, encompassing more than 21.5 million square kilometers of land grid cells. (In ELM simulations, lakes are also treated as land grid cells.)

3.2.2. Forcing data. Our experiments utilize meteorological forcing data from Daymet, which provides gridded estimates of daily weather parameters across North America from 1980 to 2014 at a 1 km spatial resolution (Kao et al., 2024; Thornton et al., 2021). The Daymet data was temporally downscaled to 3-hourly intervals to meet the requirements of km-ELM simulations. This downscaling process incorporated sub-daily temporal information from the well-established GSWP3 meteorological reanalysis dataset (Kim, 2017), ensuring consistency within each sub-daily timestep while preserving the overall daily Daymet values. Forcing samples are illustrated in the upper panels of Figure 2.

Daymet is produced by interpolating quality-controlled daily station observations (primarily from GHCN-D) to a 1 km grid over North America; additional variables such as shortwave radiation, vapor pressure, daylength, and snow water equivalent are derived diagnostically or via simple models (Thornton et al., 1997, 2020). This differs from global reanalyses like ERA5 and MERRA-2, which assimilate diverse in situ and satellite observations into numerical weather prediction systems to generate globally consistent atmospheric states at coarser native resolutions and sub-daily cadence (Gelaro et al., 2017; Hersbach et al., 2020). In our study, we use Daymet for high-resolution surface meteorology over North America, while leveraging a reanalysis (GSWP3) only to provide sub-daily temporal disaggregation consistent with the daily Daymet totals.

KiloCraft is further employed to generate one-dimensional meteorological forcing data specifically formatted for km-ELM. This process includes generating an LCC projection for geospatial transformations and performing temporal data processing. A key step involves constructing a land mask to exclude non-land areas and to assign grid cell affinity (indicating which MPI process is responsible for each grid cell). The 2D forcing data is then reshaped and extracted for valid land cells and stored in a one-dimensional format along with associated metadata. When applicable, parallel processing is used to improve efficiency.

3.2.3. Surface properties. KiloCraft is used to generate 1 km × 1 km resolution surface property data for km-ELM simulations. Users can select interpolation methods (such as nearest neighbor, linear, or spline) for individual variables to produce high-resolution data from coarse-resolution gridded datasets or collections of observational data points. Additionally, KiloCraft enables the integration of state-of-the-art data products directly into the surface inputs. In our experimental setup, we utilize a standard-resolution (0.5 ° × 0.5 °) global surface properties dataset in the WGS84 system to generate refined data products within the Daymet domain, represented in both 1D and 2D formats. km-ELM also requires additional small input files, including parameter files for ELM/CNP and snowpack treatment.

The North America domain has dimensions of 7814 × 8,075, with 21.6 million grid cells treated as land. Each grid cell is organized hierarchically, comprising various subgrid components such as topographic units, land units (up to 5 types), soil columns (up to 2 types with 15 layers each), plant functional types (up to 17 vegetation types), and cohorts. These components collectively represent the diverse characteristics of the surface and subsurface across North America. Sample outputs are shown in the bottom panels of Figure 2.

In the ELM gridcell data structure, a grid cell is divided into subgrid land units representing high-level land cover types (e.g., vegetated, lake/wetland, glacier, urban, crop). Each land unit contains one or more columns that represent unique soil-snow hydrologic columns, usually divided into 15 soil layers. Vegetation in a vegetated land unit is represented by areal fractions of plant functional types (PFTs), such as evergreen or deciduous forests, shrubs, and crops. Multiple PFT canopies can share the same column's soil/snow states while having distinct parameters. External land cover datasets are mapped to PFT fractions, and standard soil maps provide soil properties for each column. This structure aligns with traditional concepts: land units correspond to land cover units, columns to soil/snow columns, and PFTs to vegetation types in other models. (Lawrence et al., 2019; Oleson et al., 2013).

3.3. Massive km-ELM Simulation Evaluation

Accurate land–atmosphere coupling is essential at any resolution. At kilometer scale, however, several additional challenges emerge because many processes become partially resolved rather than entirely parameterized. First, parameterizations must be made scale-aware to avoid double counting or omission when topographic effects, canopy turbulence, snow processes, and surface heterogeneity are explicitly represented (Hao et al., 2022). Second,

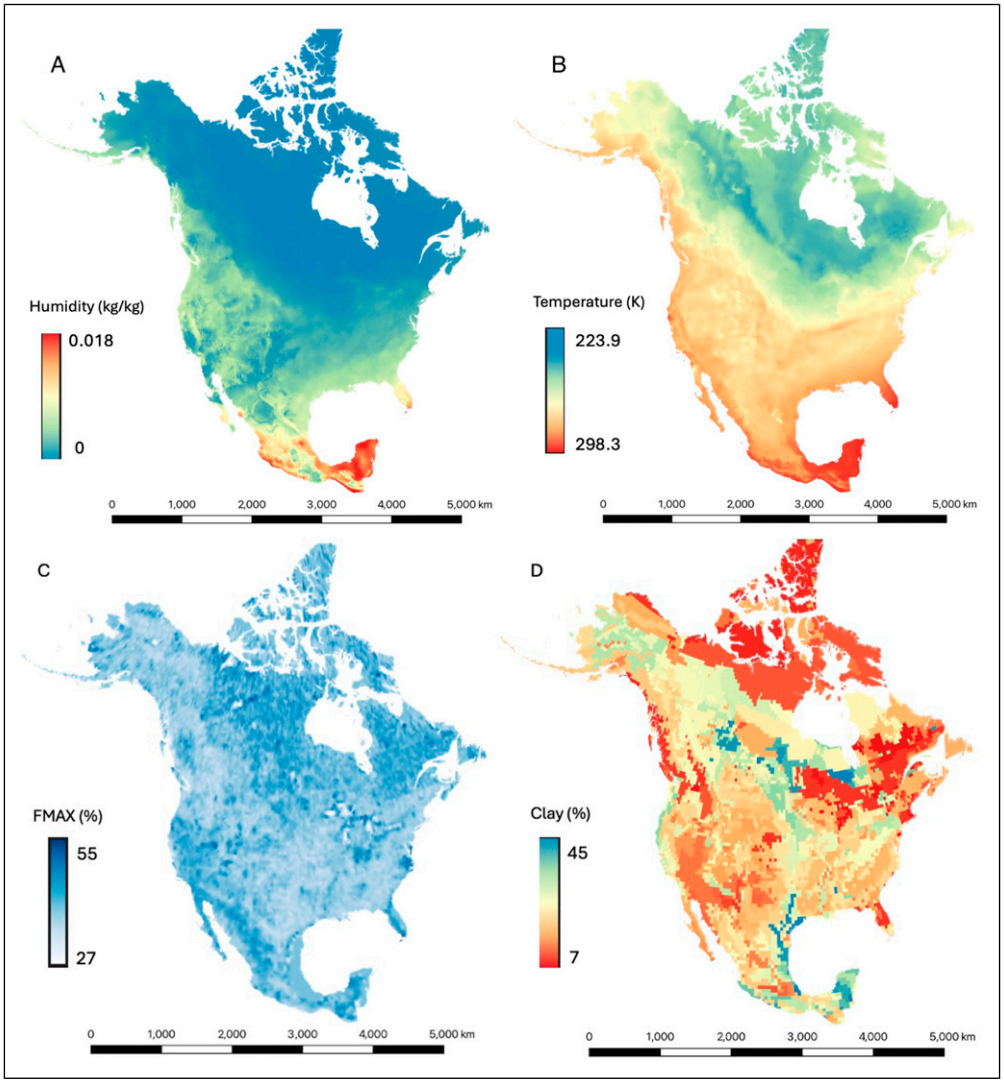


Figure 2. Samples of 1 km by 1 km input data across North America. Top left: humidity (01/01/2014); top right: temperature (01/01/2014); bottom left: maximum fractional saturated area (percentage); bottom right: clay (percentage) in soil.

sharper spatial gradients and nonlinear feedbacks at $\mathcal{O}(1\,\mathrm{km})$ demand more robust numerics, tighter coupling, and sometimes shorter timesteps. Third, lateral and mesoscale processes (e.g., hillslope runoff and subsurface flow, cold-air drainage, snow redistribution, and fine-scale disturbance/patch dynamics) gain prominence relative to coarse-grid regimes. Fourth, boundary/initial conditions and parameters must be supplied consistently at high spatial detail with quantified uncertainties, and data assimilation/validation require dense, spatially explicit observations. Finally, computational and I/O burdens increase substantially due to the explosion of subgrid tiles and outputs. These issues are in addition to the parallel computing and scalability challenges discussed above.

While efforts are ongoing to address these challenges, this work focuses on a specific study domain that demonstrated success in a previous project—the NGEE Arctic project.[4] Specifically, the ELM model over this study domain was successfully initialized through a 1000-year spin-up, followed by transient simulations from 1850 to 2014. Some resulting outputs (Snow Depth, TLAI, GPP, etc.) are consistent with observational data product and site measurements (Yuan et al., 2023). By replicating this domain, we aim to assess the scalability and input/output challenges of the km-ELM simulation framework. We first establish a collection of land grid cells in the Alaska Seward Peninsula (AKSP) region, totaling 72,083 grid cells. We then used KiloCraft to generate the required input datasets, including domain, forcing, and surface properties for the AKSP region. The AKSP experiments were conducted on the CADES `Baseline` system at Oak Ridge National Laboratory (ORNL). The open-access portion of `Baseline` is a CPU-based system. It consists of 180 nodes, each with two AMD EPYC 7713 processors, providing a total of 128 cores

running at 2.0 GHz. The AKSP experiment serves as the reference simulation for the development and evaluation of our model.

3.3.1. AKSP reference simulation. The Seward Peninsula, a landmass on the western coast of Alaska, spans approximately 330 km long and 145–225 km wide, covering 72,083 square kilometers. Table 1 lists the input data for the AKSP simulation. Figure 3 illustrates the domain and sample input variables for the AKSP reference simulation. The forcing data is a subset of the North America dataset at 1 km × 1 km spatial resolution. However, the surface properties data is interpolated using the nearest neighbor method from a coarser 0.5 ° × 0.5 ° dataset. Consequently, variables like clay soil percentage in the AKSP region show large, box-like patterns, with an effective resolution of about 50 km by 50 km. Efforts are ongoing to generate a km-scale surface properties dataset. The study examines the computational efficiency of the km-ELM using data from the AKSP region. Specifically, the km-ELM reference simulation operates around four times faster than simulations detailed in a prior study (Yuan et al., 2023) that employed a framework designed for point-wise ELM simulations in limited areas.

km-ELM evaluations are configured to produce default ELM output history files (`elm.h0`), which include all essential variables across the land grid cells. To ensure consistency across experiments and save computing resources, we use a fixed hourly timestep for a five-day simulation. The output is configured to perform temporal averaging over the entire simulation period, resulting in a single averaged timestep written at the end of the run. This setup mirrors the default ELM output configuration, in which monthly simulations yield temporally averaged variables written as a single timestep in a NetCDF file at the end of each month. The restart interval is also set to match the simulation period. As a result, at the simulation's conclusion, ELM generates restart files for all active components: the coupler (`cpl.r`), the land model (`elm.r`), and the data atmosphere model. The data atmosphere restart file is relatively small, containing only atmospheric data streams. Additionally, ELM produces a small auxiliary land restart history file (`elm.rh0`), which is necessary for resuming the simulation. This configuration represents the most demanding I/O workload scenario in ELM simulations. Table 2 summarizes information on the large output files generated in the AKSP experiment.

3.3.2. Domain duplication for simulation readiness evaluation. The experiments are currently configured as spin-up simulations that incorporate ecosystem dynamics to achieve stable-state values for subsequent transient simulations. This study highlights the capabilities of parallel simulation, kilometer-scale modeling configuration, and computational performance—including both compute and I/O efficiency. It is worth noting that ELM simulations are conducted independently at each individual land grid cell. The simulation is spatially implicit within each cell, with no lateral transfer between grid cells. Accordingly, we replicate the reference AKSP case to create numerical experiments of varying sizes, which are used to evaluate the computational performance of km-ELM on large-scale computing systems.

In our study, we replicate the AKSP case to construct ELM experiments of varying sizes, as listed in Table 3. We conduct experiments with `AKSPx10`, `AKSPx100`, and `AKSPx300`, corresponding to 10-fold, 100-fold, and 300-fold replications of the AKSP case, respectively. The largest experiment conducted (`AKSPx300`) covers a domain consisting of 21.6 million (21,624,900) land grid cells, matching the total number of land grid cells in North America as defined within the Daymet domain. This `AKSPx300` domain includes the following subgrid components: 21.6 million (21,624,900) topographic units, 93.9 million (93,936,900) land units, 353.4 million (353,435,700) soil columns, and 699.4 million (699,434,100) plant functional types.

3.4. Result validation and performance profiling

3.4.1. ELM outputs: History and restart files. The km-ELM simulations produce a single history output file along with several restart files. Output data from each file is compared to verify and validate results using duplicated input datasets. To further assess consistency, we compare the output of the AKSP case simulation on `Perlmutter` and `Summit` with that of the reference AKSP simulation on the `Baseline` system (refer to Subsection Massive km-ELM Simulation Evaluation). Since the simulations run on three different machines and most variables are stored in double precision within the history and restart files, we

Table 1. Input data files for the AKSP reference simulation.

	Num Vars	File Size	Largest dimension	Size of largest variables
Domain	15	0.09 GB	72,083 (grid cell)	4 × 72, 083
Surface data	89	1.63 GB	72,083 (grid cell)	12 × 17 × 72, 083
Forcing (annual)	7	6.01 GB	72,083 (grid cell)	248 × 72, 083

All variables are written in double precision floating point.

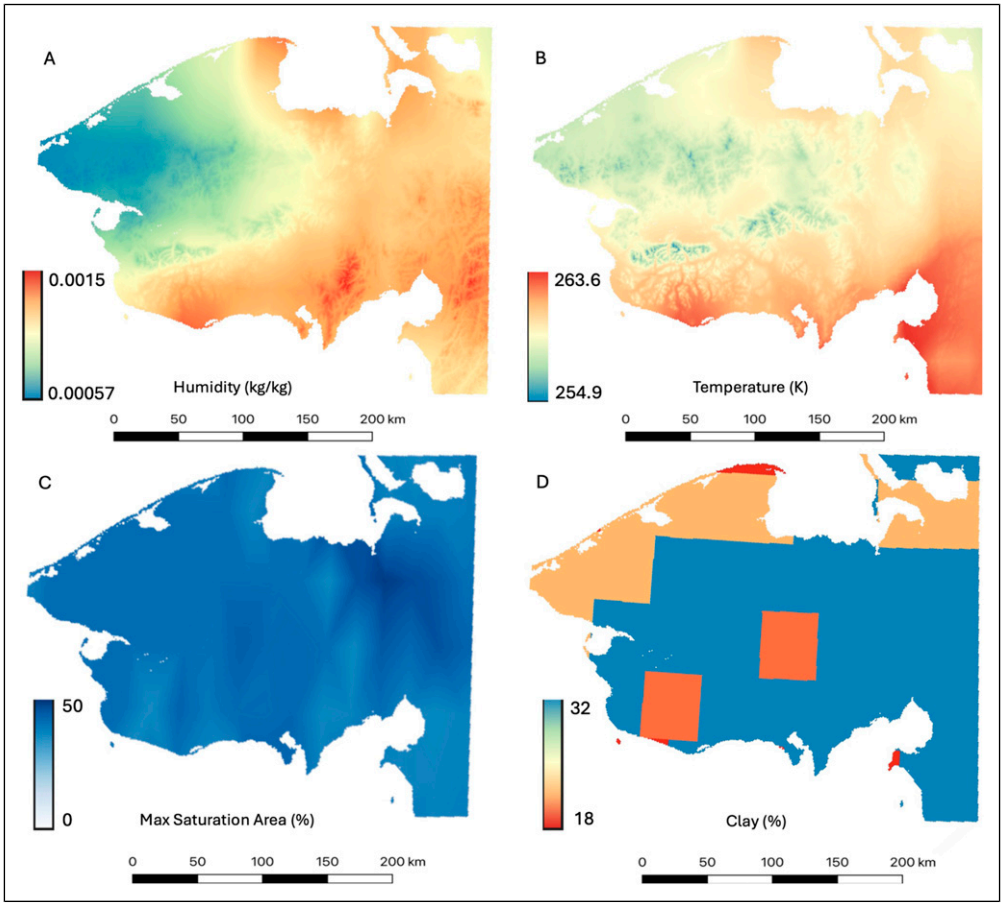


Figure 3. The domain and input dataset of the AKSP reference simulation. Top left: humidity (01/01/2014); top right: temperature (01/01/2014); bottom left: maximum fractional saturated area (percentage); bottom right: clay (percentage) in the soil.

perform a bit-by-bit comparison that accounts for machine precision to ensure equivalency at the numerical level. To support robust I/O performance, km-ELM uses the PnetCDF library and adopts the `NETCDF3_64BIT` file format.

3.4.2. Timing and performance profiling. The General Purpose Timing Library (GPTL), integrated into the CIME framework, plays a critical role in our kilometer-scale ELM experiments by collecting detailed model timing data. GPTL timers are embedded within both the CIME driver and model code to enable comprehensive performance analysis. Each model run produces three primary types of timing data: (1) overall run metrics such as total runtime, computational cost, and throughput; (2) detailed statistics from individual model timers; and (3) daily runtime tracking to identify temporal variations in performance. By leveraging GPTL, our experiments ensure precise timing evaluations, which are essential for optimizing model performance and analyzing resource utilization in kilometer-scale ELM simulations. We also use the SCORPIO capability to trace and profile I/O performance, capturing read/write data size, duration, and throughput.

To facilitate analysis, we employ the Performance Analytics for Computational Experiments (PACE) framework

Table 2. Output data files from the AKSP reference simulation.

	Num Vars	File Size	Largest dimension	Size of largest variables
elm.h0	553	0.48 GB	233,1447 (pft)	17 × 72, 083 (float)
cpl.r	92	0.05 GB	72,083 (grid cell)	1 × 72, 083 (double)
elm.r	466	14.18 GB	233,1447 (pft)	2 × 2, 331, 447 (double)

Double stands for double precision floating-point, and float means single precision.

Table 3. The size of input files of the experiments (in GB).

	AKSP	AKSP x10	AKSPx100	AKSPx300
Domain	0.01	0.1	0.9	2.8
Surface data	1.6	7.2	63.4	188.2
Forcing (annual)	7.1	50.3	497.7	1477.4

(Oak Ridge National Laboratory), which summarizes performance data collected from E3SM simulations. PACE enables the extraction of key insights and presents results through an interactive web portal. Users can explore and analyze performance data at https://pace.ornl.gov/.

4. Results and discussions

4.1. Evaluation settings

The evaluations are conducted on three clusters:

- `Baseline`: This CPU-only system consists of 180 compute nodes, each featuring two AMD EPYC 7713 processors, totaling 128 cores per node operating at 2.0 GHz, with either 256 GB or 512 GB of RAM. The system includes 2.3 PB of shared storage accessible via the Wolf2 GPFS file system.
- `Perlmutter`: The CPU partition of the `Perlmutter` comprises nodes powered by dual-socket AMD EPYC 7763 processors, providing 128 cores per node at 2.45 GHz with 512 GB of DDR4 memory. Its scratch file system is an all-flash architecture offering 44 PB of disk space, an aggregate bandwidth of 7.8 TB/s (reads) and 6.6 TB/s (writes), and 4 million IOPS (4 KiB random).
- `Summit`: `Summit` is a hybrid CPU-GPU supercomputer designed for extreme-scale scientific computing. It consists of 4608 compute nodes, each equipped with two IBM POWER9 processors (22 cores each), six NVIDIA Tesla V100 GPUs, and 256 GB of main memory. `Summit` features a 250 PB IBM Spectrum Scale file system, Alpine, with performance of approximately 2.5 TB/s for sequential I/O and 2.2 TB/s for random I/O.

In the experiments, the km-ELM model is developed using the GNU Fortran (gfortran/12.4) on `Baseline` and the classic Intel oneAPI (ifort/2021.12) compilers on `Perlmutter`. On `Summit`, the experiments are built using `ncvhp/23.9`, NetCDF (C/Fortran), and PnetCDF libraries. Additionally, two of the 44 cores per node on `Summit` are reserved for system operations, leaving 42 cores available for our experiments. Exact compiler versions and module stacks used for each system are documented in our run scripts. In practice, ELM simulations can run for extended periods, spanning up to hundreds of simulation years. The default output interval typically consists of monthly history files, with restart files saved every 10–20 years. For our evaluations, we configure the ELM simulation to run over a short 5-day period with an hourly timestep. This setup ensures that the runtime of the land component does not significantly overshadow that of the data atmosphere and coupler. Each experiment produces one history file and one restart file. These short-term simulations provide essential information for evaluating km-ELM computational capabilities and scalability within limited computational allocations. We also note that single or mixed precision could reduce memory footprint and I/O volume in some workloads; however, restarts require double precision to preserve state integrity, and precision trade-offs are beyond the scope of this study.

4.2. Simulation outputs

Our experiments produce one history file and several restart files at the end of each simulation, as detailed above. To support result validation, we develop a Python-based software utility to compare variables from the AKSP case, identifying and reporting any variables that appear in one file but are absent in another. It is important to clarify that the term ”Bit-for-Bit” (B4B) identical, as used in our work, refers to outputs that are compared exclusively within the same machine and compiler configuration. This approach aligns with the practices established in E3SM’s continuous integration (CI) system, where output baselines are maintained per machine–compiler combination. On each of the three clusters evaluated, the output data is element-wise identical, confirming numerical equivalency and adherence to established B4B standards within these controlled environments.

For duplicated AKSP cases, the utility is further used to confirm that the results precisely replicate the specified number of reference simulation copies. Figure 4 presents four output variables extracted from the history files of the AKSP case. These represent early simulation outputs from the spin-up phase, where total leaf area index values are expectedly low. Box-like patterns are visible in the results, stemming from inconsistencies in the input surface properties dataset. Nonetheless, the outcomes are identical to the reference simulation results from the `Baseline` system.

Table 4 summarizes the sizes of history and restart files generated across various experiments. The file sizes scale proportionally with the number of grid cells in each case. For instance, the `elm.h0` history file from the `AKSPx300` experiment is approximately 280 times larger than that of the baseline AKSP case. The `elm.r` restart files are the most storage-intensive, reaching 4248.54 GB at the largest scale. In contrast, `cpl.r` coupler files remain relatively small, growing from 0.05 GB to 14.20 GB. These results highlight

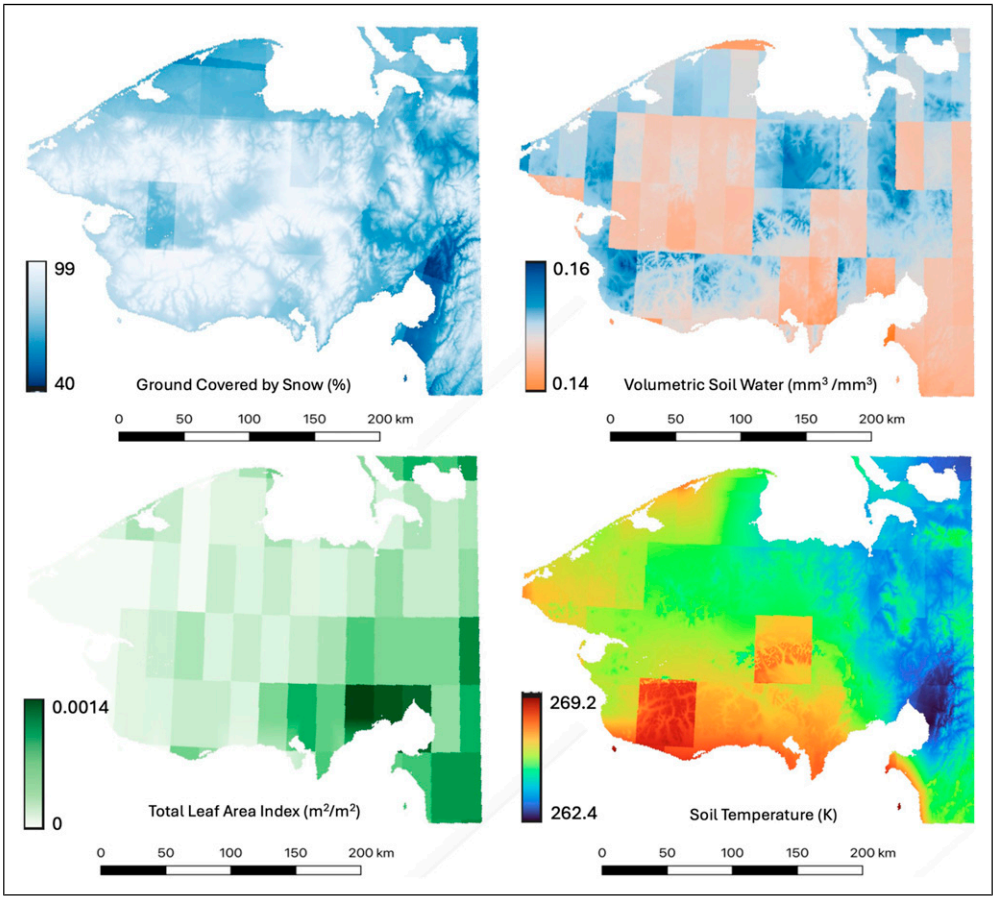


Figure 4. Visualization of output variables from the experiment. Top left: ground coverage by snow; top right: water in soil; bottom left: total leaf area index; bottom right: soil temperature.

the substantial storage and I/O burden associated with high-resolution km-ELM simulations, particularly due to restart and history files.

In addition to the SCORPIO/PnetCDF tuning explored here, several options merit further investigation for I/O performance: alternative backends (e.g., ADIOS2), substitution of the zlib backend in HDF5 with zlib-ng, and data reduction via compression and quantization. While these approaches can reduce data volume or change write characteristics, lossy methods are not suitable for restart files, and their scientific implications for history streams require careful evaluation. We therefore restrict this study to backend choice and aggregation settings and leave these options to future work.

4.3. Small-scale performance evaluations

In this study, we conduct two distinct scalability tests (strong scalability and weak scalability) using the reference AKSP simulation case, which encompasses a total of 72,083 land grid cells. The strong scalability tests evaluate how computational performance improves as the number of processing units increases while keeping the problem size constant. This helps illustrate the efficiency of our parallel implementation on larger clusters and demonstrates how well the workload is distributed across nodes with minimal overhead. In contrast, the weak scalability tests assess the system's ability to maintain performance as the problem size increases proportionally with the number of computational resources. These evaluations help determine the limits of our simulation framework in handling larger-scale scenarios and provide insight into the robustness of our approach for extensive environmental simulations.

This section presents strong and weak scalability tests at small scale on `Baseline` and `Perlmutter`, using up to 10 nodes for the `AKSP` and `AKSPx10` configurations. At this small scale, the computational cost of the data atmosphere and coupler components is negligible compared to

Table 4. Output file sizes from various experiments (in GB).

	AKSP	AKSPx10	AKSPx100	AKSPx300
elm.h0	0.48	4.81	47.91	134.18
elm.r	14.18	141.63	1416.19	4248.54
cpl.r	0.05	0.49	4.75	14.20

Table 5. Strong and weak scaling of the land model at small scale on `Baseline` and `Perlmutter`.

Cases		AKSP	AKSP	AKSPx10
Number of cores		128	1280	1280
Baseline	Time (s)	53.862	4.325	55.74
	SYPD	21.97	273.66	21.23
	Strong scaling efficiency	100%	124%	–
	Weak scaling efficiency	100%	–	97%
Perlmutter	Time (s)	52.415	4.099	53.899
	SYPD	22.58	288.74	21.96
	Strong scaling efficiency	100%	128%	–
	Weak scaling efficiency	100%	–	97%

that of the land model, so we report only the land model's performance.

Table 5 compares the wall-clock time (in seconds), simulation-years-per-day (SYPD), and scaling efficiency of the land component in km-ELM across different configurations on both the `Baseline` and `Perlmutter` systems. Three cases are evaluated: `AKSP` with 128 cores, `AKSP` with 1280 cores, and `AKSPx10` with 1280 cores. As the number of cores increases from 128 to 1280 in the `AKSP` case, both systems exhibit substantial performance improvements, with SYPD increasing more than tenfold—from 21.97 to 273.66 on `Baseline` and from 22.58 to 288.74 on `Perlmutter`. This corresponds to strong scaling efficiencies exceeding 100% on both platforms (124% on `Baseline` and 128% on `Perlmutter`), suggesting favorable superlinear scaling likely due to improved cache utilization or reduced contention. When comparing `AKSP` with 128 cores to `AKSPx10` with 1280 cores, the weak scaling efficiency remains high at 97% on both systems, demonstrating that km-ELM sustains excellent scalability as problem size increases.

4.4. Extreme-scale performance evaluations

This section extends the strong and weak scalability tests in Subsection Small-Scale Performance Evaluations to extreme scale on the `Summit` supercomputer. For the strong scalability tests on `Summit`, we use the `AKSPx300` configuration across 300, 600, 1,200, and 2400 nodes, with each node assigned 42 MPI tasks. For the weak scalability tests, we scale the domain size in proportion to the number of nodes using the following configurations: `AKSP` on 1 node, `AKSPx10` on 10 nodes, `AKSPx100` on 100 nodes, and `AKSPx300` on 300 nodes.

4.4.1. Strong scaling. The strong scaling test is conducted using `AKSPx300`, which requires a minimum of 6300 CPU cores (150 nodes) on `Summit`, resulting in approximately 3500 grid cells per core. In this setup, a fixed number of MPI processes is allocated to the data atmosphere and coupler components: 3360 processes distributed across 80 compute nodes. This allocation is necessary because the computational workload for the data atmosphere (temporal interpolation) and the coupler (data exchange) remains constant once the simulation domain is defined. The choice of 80 nodes ensures that the runtime of the atmosphere and coupler components does not dominate that of the land model during performance analysis. Table 6 summarizes the experimental configuration and reports the time spent on model initialization and execution for each component. Performance is evaluated using two metrics: wall-clock time (in seconds) and simulation-years-per-day (SYPD).

The km-ELM is a data-centric application distinguished by gridcell data structure that encompass six hierarchical levels and involve an exceptionally large number of subgrid elements (for instance, two of these, such as Plant Functional Types and cohort, each exceed two billion elements, with each element containing hundreds of multi-dimensional variables at a 1-km by 1-km resolution across North America). This immense scale induces significant memory pressure and contributes to substantial initialization overhead. The model initialization time for the `AKSPx300` cases ranges between 1600 and 2100 seconds. Initialization time is a one-time cost for each simulation, regardless of the simulation duration. In our study, the total initialization primarily influenced by the land (LND) initialization reveals that the current kmELM model initialization was not optimized for massive parallel simulation. The default initialization approach involved a single process reading the entire simulation domain information, which is then partitioned into subdomains using a round-robin scheme before distributing this subdomain information to each individual process. This step, along with the further initialization of grid cell and subgrid data structures—including substantial memory allocation—contributes significantly to the overall initalization time. Notably, this process also necessitates a synchronization step across all individual MPI processes to ensure consistency during the initialization phase.

When using 6300 cores, I/O operations noticeably impact performance. As the core count exceeds 50,400, MPI collective communication across all processes becomes a dominant factor. However, it is important to note that initialization is a one-time event at the start of the simulation, and its performance heavily depends on domain partitioning, which is beyond the scope of this study. Initialization time stabilizes around 1600 seconds when using between 12,500 and 25,200 cores, corresponding to approximately 1000 to 2000 grid cells per process. For further insights on scalable partitioning in large-scale land surface simulations (Gu et al., 2025), evaluated the impact of kmELM decomposition strategies on computation, communication, and I/O.

Table 6. Strong scaling performance. In 'ATM/CPL/LND-*a*', '*a*' denotes the number of CPU cores used.

Number of cores	6,300	12,600	25,200	50,400	100,800
Configurations	ATM- 3360 CPL- 3360 LND- 6300	ATM- 3360 CPL- 3360 LND- 12,600	ATM- 3360 CPL- 3360 LND- 25,200	ATM- 3360 CPL- 3360 LND- 50,400	ATM- 3360 CPL- 3360 LND- 100,800
Initialization (s)	1977.47	1591.55	1631.04	1963.68	2060.34
ATM time (s)	132.749	48.765	31.120	68.089	52.85
ATM SYPD	8.92	24.27	38.03	17.38	22.35
CPL time (s)	142.032	39.008	3.886	1.573	3.32
CPL SYPD	8.33	30.34	304.57	752	366.51
LND time (s)	939.447	388.043	185.725	134.960	102.042
LND SYPD	1.26	3.05	6.37	8.7	11.60

Regarding the data atmosphere (ATM), a similar performance pattern is observed, prompting further investigation into I/O behavior. Results indicate that optimal I/O performance is achieved when 25,200 cores (600 nodes) are used, with a 64 MB buffer pre-allocated for I/O operations via the PnetCDF APIs.

The performance of the coupler (CPL) improves with increasing core counts due to the use of block MPI communication for data exchange between the land and coupler components. When 100,800 cores are employed for the land component, communication overhead between the 3360 coupler cores and 100,800 land cores increases slightly. However, overall coupler execution time remains minimal when more than 25,200 cores are used in the experiments.

The land model (LND) demonstrates favorable scalability, with execution time decreasing significantly from approximately 940 seconds to 102 seconds when scaling from 6300 cores (150 nodes) to 100,800 cores (2400 nodes). Figure 5 illustrates the achieved speedup of LND. The *x*-axis represents the number of CPU cores, ranging from 6300 to 100,800, while the *y*-axis shows the speedup relative to the baseline performance at 6300 cores, normalized to 1.0. Blue bars denote ideal (linear) speedup, scaling proportionally with core count, and red bars indicate the observed speedup. Parallel efficiency—defined as the ratio of observed speedup to ideal speedup, expressed as a percentage—is annotated above each bar in the plot. As expected in strong scaling scenarios, parallel efficiency decreases as core count increases due to the fixed problem size. Notably, LND maintains high efficiency at 87% when scaling from 6300 to 50,400 cores. However, efficiency declines to 58% at 100,800 cores, where each core processes only about 220 grid cells. This drop reflects increasing overheads—including communication costs, synchronization delays, etc. (Slaughter et al., 2020)—which limit performance gains from additional compute resources.

4.4.2. Weak scaling. In the weak scaling scenario, a consistent workload of approximately 1700 grid cells is assigned to each processor. Table 7 summarizes the weak scaling results in terms of wall-clock time (in seconds) and simulation-years-per-day (SYPD). The experiments begin with the baseline configuration (`AKSP`) on 42 cores (1 node) and scale up to `AKSPx10`, `AKSPx100`, and `AKSPx300`, utilizing 420, 4,200, and 12,600 cores, respectively. The ratio of cores allocated to the atmosphere and coupler components (ATM/CPL) relative to the land model (LND) is kept consistent with the `AKSPx300` configuration, with 3360 out of 12,600 cores assigned to ATM/CPL.

It is important to note that the model initialization phase exhibits poor scalability. Initialization time increases substantially from 87 seconds in the `AKSP` case to approximately 1600 seconds in the `AKSPx300` case—an 18.4-fold increase. This degradation is primarily attributed to the initialization phase of the land model, which is a one-time cost and not scalable. In the largest configuration, the combination of a large input dataset and the use of 12,600 cores introduces significant bottlenecks in I/O operations and MPI collective communication. These issues are currently under investigation and will be addressed in future work.

During the model execution phase, the ATM component's execution time increases significantly—from 2.4 seconds in the `AKSP` case to 48.76 seconds in the `AKSPx300` case—representing a 20.3-fold increase. This

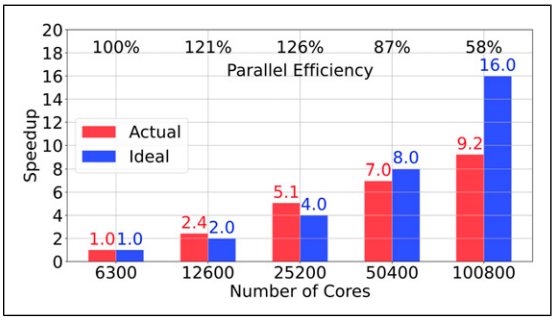


Figure 5. Speedup of LND in the strong scaling experiments (blue = ideal, red = observed).

Table 7. Weak scaling performance (blue = ideal, red = observed). In 'ATM/CPL/LND-*a*', '*a*' denotes the number of CPU cores used.

Case	AKSP	AKSPx10	AKSPx100	AKSPx300
Configurations	ATM- 11 CPL- 11 LND- 42	ATM- 112 CPL- 112 LND- 420	ATM- 1120 CPL- 1120 LND- 4200	ATM- 3360 CPL- 3360 LND- 12,600
Initialization	87.056	120.692	931.272	1591.552
ATM time (s)	2.395	11.353	28.985	48.765
ATM SYPD	494.18	104.25	40.83	24.27
CPL time (s)	0.597	6.577	8.534	39.008
CPL SYPD	1982.52	179.95	138.69	30.34
LND time (s)	316.927	374.488	392.896	388.043
LND SYPD	3.73	3.16	3.01	3.05

rise is primarily due to the much larger input dataset in the `AKSPx300` configuration. The default I/O setup utilizes between 10 and 100 nodes for read operations. Similarly, the CPL execution time increases from 0.6 seconds to 39.0 seconds, a 65.0-fold jump, primarily caused by synchronized data communication between the data atmosphere, coupler, and land components. However, the cost of ATM and CPL is negligible compared to that of LND.

In the baseline `AKSP` case—running 42 MPI tasks on a single node—the land model (LND) completes execution in approximately 317 seconds, which is more costly than on `Baseline` and `Perlmutter` (see Subsection Small-Scale Performance Evaluations), possibly due to the reduced number of available cores and the use of the `nvhpc` compiler on `Summit`. In the `AKSPx10` case, execution time increases slightly to around 375 seconds, mainly due to the additional overhead introduced by data synchronization and aggregation when transitioning from single-node to multi-node execution at each ELM simulation timestep. Notably, in larger configurations such as `AKSPx10`, `AKSPx100`, and `AKSPx300`, the land model's execution time remains stable at around 380 seconds. This consistency demonstrates effective weak scalability of LND, given the grid cell-independent nature of the land model, balanced workload partitioning, and the absence of lateral interactions among grid cells.

Figure 6 illustrates the speedup achieved in the weak scaling experiments. The *x*-axis denotes the number of cores, ranging from the baseline of 42 cores (1 node) to 12,600 cores, while the *y*-axis presents the corresponding speedup. As in Figure 5, red bars indicate ideal speedup, blue bars show measured speedup, and parallel efficiency is annotated at each scaling point. The results confirm that the land model maintains high parallel efficiency—exceeding 80% across all tested configurations—indicating strong per-core performance even as both problem size and core count increase. The minor performance degradation observed is likely attributable to current limitations in the implementation of large-scale grid partitioning and surface data structure initialization.

4.4.3. The I/O performance. One critical issue identified in this study is the limited scalability of I/O operations, particularly for write processes. In our setup, write requests

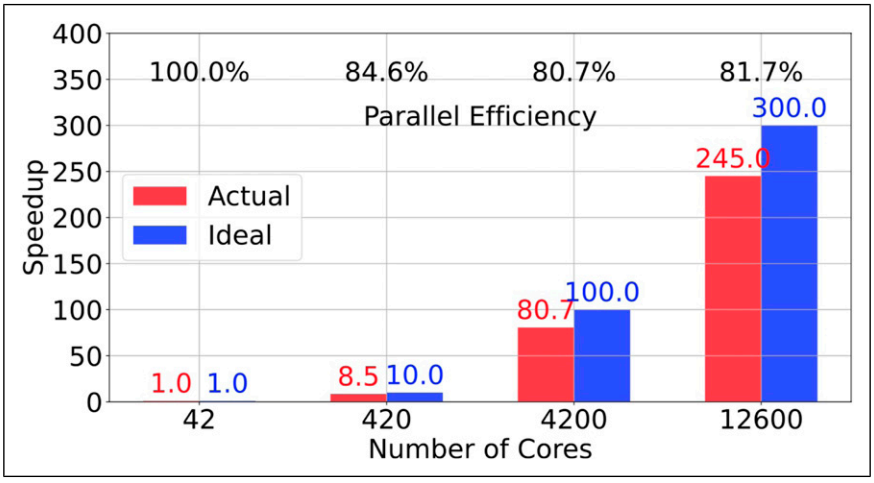


Figure 6. Speedup of LND in the weak scaling experiments.

traverse multiple software layers before interfacing with the underlying file system. Specifically, km-ELM initiates I/O operations using the SCORPIO API, which conducts internal optimizations before invoking the corresponding PnetCDF routines. PnetCDF, in turn, relies on MPI-IO for parallel I/O, ultimately issuing POSIX system calls to interact with the file system. The performance of these write operations, including both history and restart file outputs, is measured at the SCORPIO layer and recorded at the conclusion of each simulation run.

To illustrate the I/O performance, we focus our discussion on the write bandwidth observed during weak scaling experiments. In the `AKSP` case, a total of 15.14 GB of data is written in 21.51 seconds, yielding an average write bandwidth of 671.8 MB/s. For the larger configurations, `AKSPx10` and `AKSPx100`, the write bandwidths increase to 3.33 GB/s and 13.33 GB/s, respectively. In the largest case, `AKSPx300`, each process requires a substantial I/O buffer size of 64 MB to complete write operations; smaller buffers lead to write failures, which are currently under investigation. This configuration generates 4540.54 GB of data over 503.12 seconds, resulting in an average write bandwidth of 8.4 GB/s. Despite utilizing 300 compute nodes, the bandwidth scales by only 12× compared to the single-node `AKSP` run, which remains in the single-digit GB/s range—significantly below the theoretical peak bandwidth of the `Summit` file system.

Two primary factors contribute to the observed suboptimal I/O performance. First, ELM issues a high volume of small, per-process writes, which, even when aggregated by SCORPIO, result in excessive metadata overhead and file system fragmentation. Second, SCORPIO's reliance on collective I/O operations necessitates synchronization among all participating processes, leading to delays for faster processes due to stragglers, ultimately increasing total write time.

Furthermore, we explored the use of ADIOS (ADIOS2) as an alternative to PnetCDF for output management. The ADIOS framework effectively alleviates the I/O bottleneck at scale by aggregating data efficiently and enabling asynchronous/staged writes, thus improving both strong and weak scaling, increasing aggregate throughput, and reducing I/O time variability. Our results show that replacing PnetCDF with ADIOS2 (using the BP-based engine) eliminated the I/O bottleneck at leadership scale, achieving an impressive performance of up to 400 GB/s write bandwidth on 600 nodes, with end-to-end performance now dominated by computation rather than I/O (Wang et al., 2025). In summary, our findings demonstrate that through careful configuration and the adoption of advanced I/O frameworks like ADIOS2, we can significantly enhance I/O performance, ensuring that it no longer limits simulation scalability.

5. Conclusions and future work

The km-ELM is a complex application comprising over half a million lines of code, more than one thousand subroutines, and several thousand global multidimensional arrays. The successful execution of both strong and weak scaling experiments represents a significant advancement in km-ELM simulation, enabling researchers to capture the complexity and variability of terrestrial ecosystem processes across North America at kilometer-scale spatial resolution.

This study presents the first full-capacity ELM simulations at a continental scale, adopting a spatial resolution of 1 km × 1 km using up to 100,800 CPU cores. It is also worth noting that the current km-ELM implementation supports global simulations at a resolution of 3 km × 3 km. The km-ELM is fully compatible with the CIME framework and demonstrates substantial improvements in performance and scalability, addressing key challenges in data management and processing for high-resolution land modeling. Furthermore, this study provides a robust foundation to support future integration of km-ELM into fully coupled E3SM simulations. The framework can also be easily reconfigured to support km-ELM simulations over any target region, from a single land grid cell to the entire North American domain.

Moving forward, our computational efforts will focus on addressing the key performance bottlenecks identified in this study. Priorities include developing more effective domain partitioning strategies, and optimizing data structures and computational routines to reduce memory footprint and enhance execution speed. Additionally, we are exploring both lossless and lossy data compression approaches to improve I/O performance for km-ELM simulations.

The development of high-fidelity surface property datasets at kilometer scale remains a critical requirement. Future work will also involve scaling up km-ELM simulations using both CPU and GPU resources. Furthermore, advanced data analysis techniques will be developed to efficiently process and interpret the outputs of kilometer-scale ELM. Transient simulations will be conducted to capture present-day ecosystem states, and ELM land diagnostic tools, along with observational datasets, will be employed to validate model outputs.

ORCID iDs

Dali Wang https://orcid.org/0000-0001-6806-5108
Qinglei Cao https://orcid.org/0000-0002-6690-194X
Weijian Zheng https://orcid.org/0000-0003-2791-0031

Funding

The authors disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This research was supported as part of the Energy Exascale Earth System Model (E3SM) project and leverage efforts from the Next Generation Ecosystem Experiment-Arctic (NGEE-Arctic) project, funded by DOE's Office of Science, Office of Biological and Environmental Research. This research used resources of the Oak Ridge Leadership Computing Facility and Computing and Data Environment for Science (CADES) at the Oak Ridge National

Laboratory, which are supported by the DOE's Office of Science under Contract No. DE-AC05-00OR22725. This work was also partially supported by the LLNL-LDRD Program under Project No. 23-ERD-053, performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under Contract DE-AC52-07NA27344.

Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Notes

1. https://github.com/ESMCI/cime.
2. In E3SM, the "Data atmosphere model" is a software component that drives uncoupled land or ocean simulations using an atmospheric forcing dataset.
3. https://github.com/E3SM-Project/E3SM.
4. https://https://ess.science.energy.gov/ngee-arctic/.

References

Ayachit U (2015) *The Paraview Guide: A Parallel Visualization Application*. Kitware, Inc.

Burrows SM, Maltrud M, Yang X, et al. (2020) The doe e3sm v1. 1 biogeochemistry configuration: Description and simulated ecosystem-climate responses to historical changes in forcing. *Journal of Advances in Modeling Earth Systems* 12(9): e2019MS001766. https://doi.org/10.1029/2019ms001766

Caldwell PM, Mametjanov A, Qi T, et al. (2019) The doe e3sm coupled model version 1: Description and results at high resolution. *Journal of Advances in Modeling Earth Systems* 11(12): 4095–4146. https://doi.org/10.1029/2019MS001870

Commission for Environmental Cooperation (CEC) (2024) 2020 land cover of north america at 30 meters. North American Land Change Monitoring System. Canada Centre for Remote Sensing (CCRS), U.S. Geological Survey (USGS), Comisión Nacional para el Conocimiento y Uso de la Biodiversidad (CONABIO), Comisión Nacional Forestal (CONAFOR), Instituto Nacional de Estadística y Geografía (INEGI). Ed. 2.0, Raster digital data [30-m]. https://www.cec.org/north-american-environmental-atlas/land-cover-30m-2020/

Corbett P, Feitelson D, Fineberg S, et al. (1996) *Overview of the MPI-IO Parallel I/O Interface*. Springer, 127–146. DOI: 10.1007/978-1-4613-1401-1_5.

Danabasoglu G, Lamarque J-F, Bacmeister J, et al. (2020) The community earth system model version 2 (cesm2). *Journal of Advances in Modeling Earth Systems* 12(2): e2019MS001916. https://doi.org/10.1029/2019ms001916

Donahue AS, Caldwell PM, Bertagna L, et al. (2024) To exascale and beyond—the simple cloud-resolving e3sm atmosphere model (scream), a performance portable global atmosphere model for cloud-resolving scales. *Journal of Advances in Modeling Earth Systems* 16(7): e2024MS004314. https://doi.org/10.1029/2024MS004314

Dunne JP, Horowitz LW, Adcroft AJ, et al. (2020) The gfdl earth system model version 4.1 (gfdl-esm 4.1): overall coupled model description and simulation characteristics. *Journal of Advances in Modeling Earth Systems* 12(11): e2019MS002015. https://doi.org/10.1029/2019MS002015

Gelaro R, McCarty W, Suárez MJ, et al. (2017) The modern-era retrospective analysis for research and applications, version 2 (merra-2). *Journal of Climate* 30(14): 5419–5454. https://doi.org/10.1175/JCLI-D-16-0758.1

Golaz J-C, Caldwell PM, Van Roekel LP, et al. (2019) The doe e3sm coupled model version 1: Overview and evaluation at standard resolution. *Journal of Advances in Modeling Earth Systems* 11(7): 2089–2129. https://doi.org/10.1029/2018ms001603

Gu J, Klasky S, Podhorszki N, et al. (2018) Querying large scientific data sets with Adaptable IO system ADIOS. In: Yokota R and Wu W (eds) *Supercomputing Frontiers*. Springer International Publishing, pp. 51–69.

Gu Z, Wang D, Gao D, et al. (2025) Flexible user-defined domain decomposition in kilometer-scale e3sm land model simulation *Computational Science – ICCS 2025*. Springer Nature, pp. 164–178.

Han Q, Zeng Y, Zhang L, et al. (2023) Global long term daily 1 km surface soil moisture dataset with physics informed machine learning. *Scientific Data* 10(1): 101. https://doi.org/10.1038/s41597-023-02011-7

Hao D, Bisht G, Huang M, et al. (2022) Impacts of sub-grid topographic representations on surface energy balance and boundary conditions in the e3sm land model: A case study in sierra nevada. *Journal of Advances in Modeling Earth Systems* 14(4): e2021MS002862. https://doi.org/10.1029/2021ms002862

Hartnett E and Edwards J (2021) *The Parallelio (Pio) C/fortran Libraries for Scalable Hpc Performance*.

Hersbach H, Bell B, Berrisford P, et al. (2020) The era5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society* 146(730): 1999–2049. https://doi.org/10.1002/qj.3803

Kao S-C, Thornton M, Thornton PE, et al. (2024) *Gridded Sub-daily Climate Forcings for North america Based on Daymet and Gswp3 (Daymet-gswp3). Technical Report*. Oak Ridge National Laboratory (ORNL), Oak Ridge, TN (United States). Oak.

Kim H (2017) Global soil wetness project phase 3 atmospheric boundary conditions (experiment 1). Data Integration and Analysis System. (DIAS).

Krishna J, Wu D, Jacob R, et al. (2025) SCORPIO: A Parallel I/O Library for Exascale Earth System Models. https://doi.org/10.1109/IPDPSW66978.2025.00112

Lawrence DM, Fisher RA, Koven CD, et al. (2019) The community land model version 5: description of new features, benchmarking, and impact of forcing uncertainty. *Journal of Advances in Modeling Earth Systems* 11(12): 4245–4287. https://doi.org/10.1029/2018MS001583

Li J, Liao W-K, Choudhary A, et al. (2003). Parallel NetCDF: a high-performance scientific I/O Interface. In Proceedings of

the 2003 ACM/IEEE Conference on Supercomputing, SC '03, page 39, New York, NY, USA. Association for Computing Machinery. https://doi.org/10.1145/1048935.1050189

Oleson KW, Lawrence DM, Bonan GB, et al. (2013) *Technical Description of Version 4.5 of the Community Land Model (Clm). Technical Report NCAR/TN-503+STR*. National Center for Atmospheric Research.

Petersen MR, Asay-Davis XS, Barthel AM, et al. (2025). The Ocean model for E3SM global applications: Omega version 0.1.0. A new high-performance computing code for exascale architectures. *EGUsphere*, pages 1–37. https://doi.org/10.5194/egusphere-2025-3500

Puri K (2012) The australian community climate and earth system simulator, access: scientific justification and options for system development. *Australian Meteorological and Oceanographic Journal* 62: 195–210. https://doi.org/10.22499/2.6204.002

QGIS Development Team (2009) QGIS geographic information system. *Open Source Geospatial Foundation*. https://qgis.org/

Schmuck F and Haskin R (2002) GPFS: a shared-disk file system for large computing clusters. In: *Proceedings of the 1st USENIX Conference on File and Storage Technologies, FAST'02*. USENIX Association, pp. 231–244.

Schwartz P, Wang D and Thornton P (2022a) Developing an elm ecosystem dynamics model on gpu with openacc. In: *22nd International Conference on Computational Science–ICCS 2022, London, UK, June 21–23, 2022, Proceedings, Part II*. Springer, pp. 291–303.

Schwartz P, Wang D, Yuan F, et al. (2022b) Spel: Software tool for porting e3sm land model with openacc in a function unit test framework. In: *Accelerator Programming–WACCPD 2022: 9th Workshop on Accelerator Programming Using Directives, Dallas, USA, Nov 18, 2022, Proceedings*. Springer, pp. 1–14.

Sellar AA, Jones CG, Mulcahy JP, et al. (2019) Ukesm1: description and evaluation of the UK earth system model. *Journal of Advances in Modeling Earth Systems* 11(12): 4513–4558. https://doi.org/10.1029/2019ms001739

Slaughter E, Wu W, Fu Y, et al. (2020) Task bench: a parameterized benchmark for evaluating parallel runtime performance *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, pp. 1–15.

Snir M (1998) *MPI–the Complete Reference: the MPI core*. MIT press, Vol. volume 1.

Soares PMM, Johannsen F, Lima DCA, et al. (2024) High-resolution downscaling of cmip6 earth system and global climate models using deep learning for iberia. *Geoscientific Model Development* 17(1): 229–259. https://doi.org/10.5194/gmd-17-229-2024

SUN (2007). High-performance storage architecture and scalable cluster file system. Technical report, Sun Microsystems, Inc.

Thornton PE, Running SW and White MA (1997) Generating surfaces of daily meteorological variables over large regions of complex terrain. *Journal of Hydrology* 190: 214–251. https://doi.org/10.1016/S0022-1694(96)03128-9

Thornton PE, Thornton MM, Mayer BW, et al. (2020) Daymet: daily surface weather data on a 1-km grid for north America, version 4 r1. https://daac.ornl.gov/DAYMET/guides/Daymet_Daily_V4.html

Thornton PE, Shrestha R, Thornton M, et al. (2021) Gridded daily weather data for north America with comprehensive uncertainty quantification. *Scientific Data* 8(1): 1–17. https://doi.org/10.1038/s41597-021-00973-0

Wang D, Schwartz P, Yuan F, et al. (2022) Towards ultra-high-resolution e3sm land modeling on exascale computers. *Computing in Science & Engineering*(01): 1–14.

Wang D, Yuan F, Thornton PE, et al. (2023). A data toolkit for ultrahigh-resolution e3sm simulation on massively parallel processing systems. Technical report, Oak Ridge National Laboratory (ORNL), Oak Ridge, TN (United States).

Wang D, Wang C, Cao Q, et al. (2025) Scaling ultrahigh-resolution e3sm land model for leadership-class supercomputers. In: *2025 IEEE 25th International Symposium on Cluster, Cloud and Internet Computing Workshops (CCGridW)*. IEEE, pp. 1–4.

Xu Y, Wang D, Janjusic T, et al. (2017) A web-based visual analytic framework for understanding large-scale environmental models: a use case for the community land model. *Procedia Computer Science* 108: 1731–1740. https://doi.org/10.1016/j.procs.2017.05.181

Yuan F, Wang D, Kao S-C, et al. (2023) An ultrahigh-resolution e3sm land model simulation framework and its first application to the seward peninsula in alaska. *Journal of Computational Science* 73: 102145. https://doi.org/10.1016/j.jocs.2023.102145

Zheng W, Wang D and Song F (2019) Xscan: an integrated tool for understanding open source community-based scientific code. In: *International Conference on Computational Science*. Springer, pp. 226–237.

Author biographies

Dali Wang is a Distinguished Research and Development Staff member in the Environmental Sciences Division at Oak Ridge National Laboratory. He obtained his Ph.D. in Environmental Engineering (Scientific Computation) from Rensselaer Polytechnic Institute in 2002. His research areas include exascale Earth systems modeling (land), artificial intelligence/machine learning, high-performance computing, and advanced manufacturing.

Chen Wang is an Assistant Professor at the College of Computing & Data Science, Nanyang Technological University. Prior to joining NTU, he was the Fernbach Postdoctoral Fellow at the Center for Applied Scientific Computing at Lawrence Livermore National Laboratory. He received his Ph.D. in Computer Science at the University of Illinois Urbana-Champaign. His research interests include High-performance Computing, Parallel Storage System, and System for AI.

Qinglei Cao is an Assistant Professor in the Department of Computer Science at Saint Louis University. He received his Ph.D. in Computer Science from the University of

Tennessee, Knoxville in 2022, where he was advised by Dr. Jack Dongarra (Turing Award 2021) and Dr. George Bosilca. His research lies at the intersection of High-Performance Computing, Artificial Intelligence and Quantum Computing, with a focus on parallel and distributed computing, task-based runtime systems, linear algebra algorithms, large-scale machine learning and deep learning, quantum simulation, and extreme-scale scientific applications.

Jayesh Krishna is a Principal Research Software Engineer in the Mathematics and Computer Science Division at Argonne National Laboratory. He obtained his M.S. in Computer Science at Texas A&M University in 2006. His research areas include high-performance computing, I/O libraries, task and data parallel tools for Earth system modeling and software engineering.

Weijian Zheng is an Assistant Computer Scientist in the Data Science and Learning Division at Argonne National Laboratory. He obtained his Ph.D. in Computer Science from Purdue University in 2022. His research areas include AI for science, high-performance computing, distributed systems, parallel machine learning, and large language models.

Peter Schwartz is a Software Engineer in the Environmental Sciences Division of Oak Ridge National Laboratory. He received his M.S. degree in Physics from the University of Tennessee, Knoxville. His research interests include Earth system modeling, tools for understanding legacy code, and sustainably porting to new architecture.

Fengming Yuan is a Professional Technical Staff in the Environmental Sciences Division at the Oak Ridge National Laboratory. He received his Ph.D. in Soil Sciences in 2003 from the University of Wisconsin, Madison. His research interests include supercomputing application in modeling complexity of water, energy, carbon and nitrogen cycling in agricultural, forest and other terrestrial ecosystems, at various scales (field, watershed, regional and global).

Daniel Ricciuto is a Distinguished Research and Development Staff member in the Environmental Sciences Division, Oak Ridge National Laboratory, USA. He received his Ph.D. in Meteorology in 2006 from the Pennsylvania State University. His research interests include Earth system modeling and the application of uncertainty quantification (UQ) techniques to land-surface models with the goal of improving model algorithms, parameterization and predictive skill at multiple spatial scales using ecosystem observations.

Peter Thornton is a Corporate Fellow in the Environmental Sciences Division at Oak Ridge National Laboratory, USA. He received his Ph.D. in Forestry from the University of Montana. His research interests include Earth system modeling, coupled biogeochemical cycles in land ecosystems, and human–Earth system interactions.

Shih-Chieh Kao is a Distinguished Research and Development Staff member in the Environmental Sciences Division at Oak Ridge National Laboratory. He obtained his Ph.D. in Hydraulic and Hydrologic Engineering from Purdue University in 2008. His research areas include high-performance computing, hydrologic modeling, flood simulation, hydro-climate impact assessment, and hydropower resource evaluation.